Bro, listen — let's document VEERA properly so that you know exactly what you've completed, what is still missing, and what your friend needs to do from the ZIP all the way to the finished project. ❤️

I'm going to separate the work we've actually completed from the components we've only prototyped or discussed. That distinction matters for your research report and for handing the project to your friend.

Part 1 — VEERA project status

1. What VEERA is supposed to do

VEERA — Failure-Aware Visual Evidence Acquisition and Adaptive Compute Allocation for Reliable Vision-Language Reasoning

The intended system takes an image and a natural-language question, answers using visual reasoning, estimates whether that answer is reliable, decides whether extra visual evidence is worth the cost, allocates compute appropriately, and either returns an answer or requests review when uncertainty remains.

Input: image + question

Vision-language model

Fine-tuned Qwen2.5-VL-3B + LoRA

Reliability and evidence controller

Decide whether the initial evidence is sufficient

Sufficient evidence

Return an answer

Insufficient evidence

Inspect another region if worthwhile

Final disposition

Answer, inspect again within budget, or abstain/request review

The important point is that we have trained the vision-language model, but we have not yet completed and validated the entire adaptive controller as one integrated system.

2. Detailed status of every component

I'll use four statuses:

- Done
  — completed and supported by an actual run or saved result.
- Prototype
  — implemented or tested experimentally, but not yet validated as part of the complete system.
- Next
  — needed to finish the project.
- Optional
  — an enhancement, not a blocker for a basic demo.

A. Base vision-language model

Pretrained Qwen2.5-VL-3B-Instruct loaded

The pretrained 3B-parameter model ran successfully on a Kaggle Tesla T4.

Image and question processing

A real image, a question and a yes/no answer were processed together. The multimodal forward pass succeeded.

General visual question answering

The completed fine-tuning experiment covers yes/no questions. Open-ended answers, counting, spatial reasoning and other VQA tasks have not been comprehensively evaluated.

B. Dataset and experimental splits

RePOPE training data prepared

2,867 image-question-answer examples from 300 images, with no missing question/answer values in the checked training CSV.

Image-disjoint validation and test sets

Validation: 954 questions / 100 images. Test: 958 questions / 100 images. These images do not overlap with training or each other, according to the recorded split setup.

External-dataset generalization

The GQA experiments were performed with the earlier baseline/controller. The newly fine-tuned adapter has not yet been comprehensively evaluated on GQA or another independent benchmark.

C. Fine-tuning

LoRA adapters attached and trained

Rank 8, alpha 16, dropout 0.05; attention projections `q_proj`, `k_proj`, `v_proj`, `o_proj`; one epoch and 180 optimizer steps.

Training completed without GPU memory failure

Final reported running mean loss: 0.0470. Peak reported GPU allocation: approximately 7.22 GB.

Adapter saved

`/kaggle/working/veera_lora_final`, with `adapter_model.safetensors` of approximately 14.10 MB and processor/tokenizer assets.

D. Evaluation

Baseline compared with fine-tuned model

The same validation and test examples were evaluated using the saved adapter. Prediction CSVs were saved.

Paired validation error analysis

22 baseline errors were corrected and 4 correct baseline answers became wrong; net gain 18.

Statistical uncertainty and broader robustness

We haven't completed a formal confidence-interval/significance analysis for the paired improvement or broad robustness testing. The results are promising, but should not be described as universal performance.

E. Reliability and visual evidence acquisition

Initial reliability heuristic

A yes/no probability-gap heuristic and an abstain/review disposition were implemented and tested with the earlier baseline pipeline.

Crop-inspection experiments

Additional image crops were tested. Results varied; some experiments corrected answers, others broke correct answers. Cropping is not yet proven to improve accuracy reliably.

Adaptive evidence controller for the fine-tuned model

Not yet validated end to end. We need to measure whether an additional crop actually helps, when the system should stop, and when it should abstain.

F. Adaptive compute and deployment

Automatic CPU/GPU allocation

Not implemented or validated as an adaptive scheduler. The model was run on the Kaggle GPU. A rule-based scheduler can be built without training another model.

Release ZIP created

`VEERA_finetuned_release.zip`, 15.1 MB, containing adapter assets, evaluation predictions and an experiment summary. Download completion has not been confirmed yet.

Local deployment and UI integration

Not yet verified. Your friend is expected to handle the local runtime and UI integration.

3. The measured results your friend should preserve

Metric	Original baseline	Fine-tuned VEERA
Validation accuracy	93.61% (893/954)	95.49% (911/954)
Test accuracy	93.74% (898/958)	95.82% (918/958)
Test errors	60	40
Mean test latency	~0.439 s	0.503 s

The fine-tuned model gained 20 correct predictions on the held-out RePOPE test set. These numbers are for the tested yes/no setup, not for the complete evidence-acquisition controller.

Part 2 — Exactly what is inside the ZIP?

Your friend should understand this before writing any code.

The archive is at `/kaggle/working/VEERA_finetuned_release.zip` in your Kaggle session. It is 15.1 MB.

Included in the release ZIP	Purpose
`adapter/adapter_model.safetensors`	The learned LoRA weights

`adapter/adapter_config.json`	Describes the adapter and its configuration
`adapter/` processor and tokenizer files	Files needed to process inputs and format conversations
`results/veera_finetuned_validation.csv`	Fine-tuned validation predictions
`results/veera_finetuned_test.csv`	Fine-tuned test predictions
`results/veera_validation_baseline.csv`	Original validation predictions
`results/veera_test_baseline.csv`	Original test predictions
`experiment_summary.json`	Configuration and recorded evaluation metrics
`README.md`	Release notes and limitations

The ZIP does not include:

- The complete pretrained Qwen2.5-VL-3B base-model weights.
- The complete training dataset and image files.
- Your entire Kaggle notebook and every intermediate experiment output.
- A finished backend, UI, or validated adaptive controller.

So your friend should treat the ZIP as a trained adapter release, not as a complete standalone model.

Part 3 — Full handoff guide for your friend

This is the order I recommend following. Your friend should complete each stage and verify it before moving to the next one.

Step 1 — Download and inspect the release

1. Open the Kaggle notebook where you created the ZIP.
2. Open the notebook's output/files panel.
3. Download `VEERA_finetuned_release.zip`.
4. Extract it into a dedicated project directory.
5. Confirm that `adapter/adapter_model.safetensors` exists and is not empty.
6. Confirm the four baseline/fine-tuned prediction CSVs and `experiment_summary.json` are present.

The directory should look similar to this:

```
VEERA/
├── release/
│   ├── adapter/
│   │   ├── adapter_model.safetensors
│   │   ├── adapter_config.json
│   │   ├── tokenizer.json
│   │   ├── tokenizer_config.json
│   │   └── ...processor files
│   ├── results/
│   │   ├── veera_finetuned_validation.csv
│   │   ├── veera_finetuned_test.csv
│   │   ├── veera_validation_baseline.csv
│   │   └── veera_test_baseline.csv
│   ├── experiment_summary.json
│   └── README.md
└── ...backend and UI to be created
```

The precise extraction layout can vary depending on the ZIP tool. The important thing is that the adapter files stay together.

Step 2 — Obtain the base model

Your friend needs the original pretrained model:

Qwen2.5-VL-3B-Instruct

Official model page: Qwen/Qwen2.5-VL-3B-Instruct on Hugging Face.

The base model and your adapter are two separate things:

- Base model = pretrained visual and language capabilities.
- LoRA adapter = the learned changes from your fine-tuning run.

The adapter is small because it contains only the additional trainable weights. It cannot replace the base model.

Your friend should download/cache the base model through Hugging Face Transformers and keep its location or model ID in the project configuration.

Step 3 — Set up a local Python environment

If your friend is using your MacBook Air M1, I'd begin with a separate Python virtual environment rather than modifying the system Python.

For example, in Terminal:

```
mkdir VEERA
cd VEERA

python3 -m venv .venv
source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install torch transformers peft accelerate pillow pandas
```

These are starting dependencies, not a guarantee that every newest package combination will work. Your successful Kaggle environment used PyTorch 2.11.0, PEFT 0.20.0, Transformers 5.16.1 and Accelerate 1.14.0; the local Mac uses a different hardware backend, so your friend should record and verify the installed versions.

Important for the M1: the base model is approximately 3 billion parameters.
Half-precision weights alone have a theoretical size of roughly 6 GB decimal (about 5.6 GiB), before accounting for the vision tower, runtime overhead, temporary tensors, and KV cache. A 16 GB Mac may be able to run it, but memory pressure and speed must be tested.

Step 4 — Load the base model and adapter together

The inference program should:

1. Load the base Qwen model.
2. Load the adapter from the extracted release directory.
3. Load the processor/tokenizer.
4. Move the model to the selected device.
5. Accept an image and a question.
6. Generate a yes/no answer.

A first local inference script can look like this:

import torchfrom PIL import Imagefrom peft import PeftModelfrom transformers import (AutoProcessor, Qwen2_5_VLForConditionalGeneration,)BASE_MODEL = "Qwen/Qwen2.5-VL-3B-Instruct"ADAPTER_DIR = "release/adapter"# Select Apple's Metal backend when available.if torch.backends.mps.is_available(): device = torch.device("mps") dtype = torch.float16else: device = torch.device("cpu") dtype = torch.float32processor = AutoProcessor.from_pretrained(ADAPTER_DIR)base_model = Qwen2_5_VLForConditionalGeneration.from_pretrained(BASE_MODEL, torch_dtype=dtype, low_cpu_mem_usage=True,)model = PeftModel.from_pretrained(base_model, ADAPTER_DIR)model = model.to(device)model.eval()def veera_answer(image_path, question): image = Image.open(image_path).convert("RGB") messages = [{

This is a starting script, not yet a verified Mac deployment. If loading fails due to processor files, library versions, unsupported MPS operations or memory, your friend should troubleshoot that before connecting the UI. Avoid silently switching to a different base model or quantized variant, since that can change the experimental model and its behavior.

Step 5 — Build the reliability and evidence controller

This is the most important remaining research component.

Your friend should not simply generate an answer and label it “high confidence.” The controller should decide whether the evidence is sufficient, whether another visual inspection is worthwhile, and whether to stop.

Start with the existing heuristic rather than training a new model.

Proposed controller logic

A. Initial inference

Answer + yes/no probability signals + inference cost

B. Estimate reliability

Use the existing probability-gap heuristic, then calibrate it on validation data

Reliable enough

Return the answer if the stopping rule is satisfied.

Uncertain

Estimate whether another crop could provide useful evidence.

C. Inspect, stop or abstain

Use a crop only within the compute budget; otherwise answer with appropriate uncertainty or request review.

How to implement it

1. Expose reliability signals from the model. Record the yes/no token probabilities and their gap. Treat the gap as a heuristic, not a calibrated correctness probability.
2. Run the first prediction. Save the answer, reliability signal and latency.
3. Decide whether to inspect more. Begin with the existing threshold-based policy and compare it with always answering immediately.
4. Inspect a crop when appropriate. Use the existing crop-generation helper, but ensure the crop actually contains the relevant visual region where possible.
5. Compare the evidence. If the answer changes or the evidence remains contradictory, don't automatically trust the newest answer.
6. Stop or abstain. Define a maximum number of inspections and a compute/time budget.

7. Record the decision trace. Log the original answer, each inspected crop, changed answers, total latency, final answer and whether the system abstained.

The existing experiments showed that crops sometimes helped and sometimes hurt. So the initial goal is not to claim that more inspection always improves accuracy. It is to find out when the additional cost is justified.

What should be measured?

- Accuracy when the system answers.
- Coverage: the percentage of questions it answers instead of abstaining.
- Error rate among answered questions.
- How often extra crops correct or break answers.
- Additional inference time and compute cost per question.
- Whether the controller catches errors without flagging too many correct answers.

The thresholds must be chosen using validation data. Keep the test set out of controller tuning.

Step 6 — Implement CPU/GPU scheduling

This component does not require another neural network to get started.

The first version can use a simple policy:

Condition	Initial scheduling policy
Image loading and resizing	CPU
GPU available with sufficient memory	GPU for vision-language inference
GPU unavailable	Attempt CPU inference if memory and latency permit

GPU memory insufficient	Reject gracefully, use a smaller/quantized deployment variant after separate validation, or request a retry
Extra crop has low expected value	Do not spend extra inference compute

Your friend should build a small resource-management module with inputs such as device availability, available memory, measured inference latency, and remaining compute budget.

Don't move the 3B model between CPU and GPU for every question. That can create expensive transfers and memory pressure. Normally, load the model onto a selected device and keep it there for the session. Use the controller to decide how much inference work to do, while the scheduler handles hardware availability.

Later, if a learned compute policy proves useful, you can train it using logged decisions and measured outcomes. That is optional—not a prerequisite for the first working version.

Step 7 — Build the backend and connect the UI

Your friend is responsible for integrating the model with the UI. I'd keep the architecture simple at first.

```
VEERA/
├── release/
│   ├── adapter/
│   └── results/
├── backend/
│   ├── app.py               # API endpoints
│   ├── model_loader.py      # Load base model + adapter
│   ├── inference.py         # Image + question → answer
│   ├── reliability.py       # Reliability signals
│   ├── evidence_controller.py # Inspect, stop, or abstain
│   ├── resource_scheduler.py  # CPU/GPU policy
│   └── schemas.py           # Request/response structures
├── frontend/
│   ├── index.html
```

```
│   ├── style.css
│   └── script.js
├── tests/
├── requirements.txt
└── README.md
```

This is a proposed structure for the completed project, not a claim that these files already exist.

Backend responsibilities

The backend should:

1. Accept an uploaded image and question.
2. Validate the image and question.
3. Load the model once at startup rather than reloading it for every request.
4. Run initial inference.
5. Call the reliability and evidence controller.
6. Return the final answer, uncertainty/review status, and relevant evidence-inspection details.
7. Log latency and resource usage for evaluation.

A possible API response is:

<escape>{
  "answer": "yes",
  "status": "answered",
  "reliability_score": 0.0,
  "evidence_inspections": 0,
  "latency_seconds": 0.0,
  "reason": "Initial evidence was sufficient"
}</escape>

Those numbers are placeholders, not actual model outputs. The reliability score should only be represented as a probability if it has been calibrated and validated. Otherwise, expose it as a heuristic score with clear documentation.

Frontend responsibilities

The UI should provide:

- An image upload control.

- A question input.
- An “Analyze” button.
- The answer and uncertainty/review status.
- An optional display of the crops that were inspected.
- Processing time and number of inspections.
- A graceful error message if the model cannot run.

The frontend should call the backend API rather than loading the 3B model directly in the browser.

If your friend uses HTML/CSS/JavaScript for the UI, they can keep the frontend independent of the Python inference backend.

Step 8 — Test the integrated application

Before calling VEERA complete, your friend should run these checks.

Integration checklist

0 of 10

Model loading

Base model and LoRA adapter load together without missing or unexpected adapter errors.

Basic prediction

A known image-question pair produces a yes/no answer.

Invalid input

Missing images, unsupported file types and empty questions return clear errors.

Reliability path

The response includes the documented reliability signal and status.

Additional evidence

The controller records crop inspections and respects its maximum inspection budget.

Abstention

Unresolved or contradictory evidence can lead to review rather than a forced confident answer.

CPU/GPU behavior

The selected device is reported accurately; unavailable devices and memory failures are handled gracefully.

Latency and memory

Measure total response time and peak memory for the complete pipeline.

UI integration

Upload → question → answer/review status works end to end.

Reproducibility

Record the base-model ID, adapter version, package versions and controller configuration.

Copy checklist

Step 9 — Complete the research evaluation

A working UI does not, by itself, establish that the research contribution works. The complete project should compare the following configurations:

Configuration	What it tells you
Original pretrained model	Reference performance before fine-tuning
Fine-tuned model, one pass	Benefit of task-specific fine-tuning
Fine-tuned model + reliability threshold	Whether selective answering reduces errors

Fine-tuned model + crop inspection	Whether additional evidence helps
Full VEERA controller	Whether the integrated policy improves the accuracy–coverage–compute trade-off

Evaluate on held-out examples, report confidence intervals where appropriate, and avoid tuning thresholds on the final test set. Include examples where crops help, crops hurt, and the system correctly abstains.

For the CPU/GPU component, report the selected device, latency, memory use and any fallback behavior. A rule-based scheduler is a valid initial implementation, provided you describe it accurately and measure its behavior.

Step 10 — Prepare the final deliverables

By the end, your friend should have:

- A runnable local inference environment.
- The downloaded adapter and the compatible base model.
- A backend and connected UI.
- The reliability and evidence controller.
- A CPU/GPU scheduling policy.
- Logs and test results for the integrated system.
- A reproducible environment and setup instructions.
- A research report documenting methods, baselines, ablations, limitations and results.
- A copy of the original fine-tuned adapter and predictions preserved as the reference version.

Final summary: what you should do now

You: finish the handoff

Download the 15.1 MB release ZIP and give it to your friend. Preserve the Kaggle notebook and original result files too.

Your friend: make the model run locally

Get the base model, load the adapter, and verify one image-question prediction before building the UI.

Together: finish the adaptive system

Integrate reliability, evidence inspection, stopping/abstention and resource scheduling; then evaluate the complete workflow.

My most important recommendation: don't overwrite or retrain the current adapter while your friend is integrating it. Keep this 95.82%-test-accuracy checkpoint as your reference version. Any later changes to the controller or model should be saved as a new version and evaluated separately.